

КРИПТОГРАФІЯ

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №2

Криптоаналіз шифру Віженера

ФБ-43 Цвілій Влада

Варіант 23

Мета роботи

Засвоєння методів частотного криптоаналізу. Здобуття навичок роботи та аналізу поточкових шифрів гамування адитивного типу на прикладі шифру Віженера

```
1  import os
2  from collections import Counter
3  # --- Константи та алфавіт ---
4  ALPHABET = "абвгдежзийклмнопрстуфхццщщъьэюя"
5  M = len(ALPHABET)
6  CHAR_TO_IDX = {ch: i for i, ch in enumerate(ALPHABET)}
7  IDX_TO_CHAR = {i: ch for i, ch in enumerate(ALPHABET)}
8
9  # --- Функції обробки тексту та шифрування ---
10 def read_file_safely(candidates=["text.txt", "TEXT", "text.txt.txt"]):
11     target_file = None
12     for c in candidates:
13         if os.path.exists(c):
14             target_file = c
15             break
16
17     if not target_file:
18         for f in os.listdir("."):
19             if f.endswith(".txt"):
20                 target_file = f
21                 break
22
23     if not target_file:
24         raise FileNotFoundError("Файл не знайдено")
25
26     for enc in ["windows-1251", "utf-8", "cp1251", "utf-8-sig"]:
27         try:
28             with open(target_file, "r", encoding=enc) as f:
29                 content = f.read()
30                 cleaned = clean_text(content)
31                 if len(cleaned) > 1000:
32                     return cleaned
33         except Exception:
34             continue
35
36     raise ValueError("Помилка зчитування файлу")
37
```

```

39 def clean_text(raw_text: str) -> str:
40     raw_text = raw_text.lower().replace("ë", "e")
41     return "".join(ch for ch in raw_text if ch in CHAR_TO_IDX)
42
43
44 def index_of_coincidence(text: str) -> float:
45     n = len(text)
46     if n <= 1:
47         return 0.0
48     counts = Counter(text)
49     numerator = sum(count * (count - 1) for count in counts.values())
50     return numerator / (n * (n - 1))
51
52
53 def vigenere_encrypt(plaintext: str, key: str) -> str:
54     key = clean_text(key)
55     res = []
56     r = len(key)
57     for i, ch in enumerate(plaintext):
58         x = CHAR_TO_IDX[ch]
59         k = CHAR_TO_IDX[key[i % r]]
60         res.append(Idx_To_Char[(x + k) % M])
61     return "".join(res)
62
63
64 def vigenere_decrypt(ciphertext: str, key: str) -> str:
65     key = clean_text(key)
66     res = []
67     r = len(key)
68     for i, ch in enumerate(ciphertext):
69         y = CHAR_TO_IDX[ch]
70         k = CHAR_TO_IDX[key[i % r]]
71         res.append(Idx_To_Char[(y - k) % M])
72     return "".join(res)
73

```

```

74
75 def coincidence_count(ciphertext: str, r: int) -> int:
76     return sum(
77         1 for i in range(len(ciphertext) - r) if ciphertext[i] == ciphertext[i + r]
78     )
79
80
81 def avg_block_ic(ciphertext: str, r: int) -> float:
82     blocks = [ciphertext[i:r] for i in range(r)]
83     ics = [index_of_coincidence(b) for b in blocks if len(b) > 1]
84     return sum(ics) / len(ics) if ics else 0.0
85
86
87 VARIANT_3_CIPHERTEXT = clean_text("""
88 ебюятфхмпнякнпцщявпрымтчкктьлвацхтжшэргущнныюкшяпътшюмвзщъэввачъймучицъхцщъдерэхшълдунхтутс
89 ьэхъыибгмттэбгбптщныоасякдущйпюшоибаужеуацебаъпдвхцюбхуюкыфйнбэнощопыльышдяхнцххктнкащовацъб
90 тощечйишсъчятаеюэюшъаърнчхшъфйтъккциннчсуйгбошрчызхтъюкщдшоцеаъшбнштцъщчylumцзаънэюбыеуच्या
91 ющдтновъьцртшъцъжыытекъстптшрхтфегоэзсссфажгьифюрньокяьхкщяйэвъушешчърймуолььрннхычшысюзщютз
92 фычшыбрылцбырдцъкцойупъууукояиьжууыуляуьосятпцбашяптымиаашнпцапрнпъснмнвфдшоцкяоемаящъьешезтш
93 ьеоэтхтучмыжаоемаящъьуляпъоцтмарцтыяпювчлтпхачвдъцфтячаоютьпешчфпаоепъдхшеетшяктъасяылюбъыьыо
94 екптъхххкшнэсмешчмпчфюбалчоцомитцъщылушфнзъпчыеекылмснмацъжббшефюспкчърйбуяьбйзфйрсьцоауйакт
95 шьмлтрхтжаечоьоникъфивгмьомыйчаддчцфайогпщсзмашъышгдрвоьазаоныгшбцякуювдйъцжпорерущиюпяцяъьоьв
96 аякяъшннуидвхккпдвтйшдбъкошъоьспупбыпъьуьизыятжбъоьчуьырндхкщдшбцпсоцомебыфакэншафвоашцнфшуй
97 ээююфхъжетщъпшьясаьцщмпыкечоптгяцъюиплуаьчдйгущыэнтъждягуюэшыузысрягзряшчечуоеращцубыцк

```

```

145 def main():
146     # --- Підготовка даних та частот ---
147     full_cleaned = read_file_safely()
148     n_total = len(full_cleaned)
149     counts_full = Counter(full_cleaned)
150     rus_freqs = {ch: counts_full[ch] / n_total for ch in ALPHABET}
151     i_theor = sum(p**2 for p in rus_freqs.values())
152     i_rand = 1.0 / M
153
154     print("=== ПІДГОТОВКА ДАНИХ ===")
155     print(f"Символів відкритого тексту: {n_total}")
156     print(f"I_теор = {i_theor:.5f}")
157     print(f"I_випадк = {i_rand:.5f}\n")
158

```

Підготовка даних та допоміжні функції:

Здійснюється завантаження відкритого тексту з практикуму №1 з автоматичним підбором кодування (Windows-1251/UTF-8) та його нормалізація під алфавіт із 32 літер (без «ё» та розділових знаків). За отриманим текстом обчислюються еталонні частоти літер, теоретичний індекс відповідності мови ($I_{\text{теор}}$) та рівень шуму ($1/32$). Також визначено базові функції шифрування/дешифрування Віженера, розрахунку індексу відповідності та кількості збігів (D_r).

1. Дослідження шифру Віженера

```

158 # --- Розділ 1: Дослідження шифру Віженера ---
159 print("=== ЧАСТИНА 1. ДОСЛІДЖЕННЯ ШИФРУ ВІЖЕНЕРА ===")
160 sample = full_cleaned[: min(n_total, 3000)]
161 ic_sample = index_of_coincidence(sample)
162
163 test_keys = ["да", "мир", "лето", "книга", "криптографія"]
164
165
166 print(
167     f"{'Текст':<18} | {'Ключ':<15} | {'Довжина r':<10} | {'Індекс I':<10}"
168 )
169 print("-" * 60)
170 print(f"{'Відкритий':<18} | {'-':<15} | {'-':<10} | {ic_sample:.5f}")
171
172 for k in test_keys:
173     ct = vigenere_encrypt(sample, k)
174     ic_val = index_of_coincidence(ct)
175     print(f"{'Шифртекст':<18} | {k:<15} | {len(k):<10} | {ic_val:.5f}")
176 print("-" * 60 + "\n")
177

```

Використані ключі:

```
test_keys = ["да", "мир", "лето", "книга", "криптография"]
```

```

=== ПІДГОТОВКА ДАНИХ ===
Символів відкритого тексту: 2362361
I_теор = 0.05578
I_випадк = 0.03125

=== ЧАСТИНА 1. ДОСЛІДЖЕННЯ ШИФРУ ВІЖЕНЕРА ===
Текст          | Ключ          | Довжина r    | Індекс I
-----
Відкритий      | -             | -            | 0.05614
Шифртекст     | да           | 2            | 0.04362
Шифртекст     | мир          | 3            | 0.03967
Шифртекст     | лето        | 4            | 0.03818
Шифртекст     | книга       | 5            | 0.03713
Шифртекст     | криптография | 12           | 0.03307

```

Результати розрахунку індексів відповідності:

Текст / Шифртекст	Ключ	Довжина ключа r	Індекс відповідності I
Відкритий текст	—	—	0,05614
Шифртекст	да	2	0,04362
Шифртекст	мир	3	0,03967
Шифртекст	лето	4	0,03818
Шифртекст	книга	5	0,03713
Шифртекст	криптографія	12	0,03307

Теоретичний індекс для досліджуваної мови: $I_{\text{теор}} = 0,05578$

Теоретичний індекс для випадкового шуму ($1/32$): $I_0 \approx 0,03125$

Порівняння та аналіз результатів:

Відкритий текст: Значення індексу становить $I = 0,05614$, що практично збігається з теоретичним значенням мови ($0,05578$). Це пояснюється нерівномірним розподілом літер у природній мові: наявність частих літер («о», «е», «а») суттєво підвищує ймовірність випадкового збігу двох символів.

Вплив шифрування: Шифр Віженера є поліалфавітним, тому різні символи відкритого тексту зміщуються на різні значення залежно від поточної літери ключа. Внаслідок цього однакові літери відкритого тексту переходять у різні літери шифртексту, що призводить до згладжування природних частотних піків.

Залежність від довжини ключа: Зі збільшенням періоду ключа r від 2 до 12 спостерігається чітке спадання індексу відповідності від $0,04362$ до $0,03307$. Чим більша довжина ключа, тим сильніше перемішуються статистичні характеристики мови і тим ближчим стає шифртекст до випадкової послідовності, де теоретичний індекс прямує до межі $1/32 \approx 0,03125$.

Отже, шифр Віженера приховує статистичні закономірності мови тим краще, чим довшим є ключ. Через це прямий частотний аналіз до всього шифртексту застосувати неможливо — його спочатку необхідно розділити на окремі моноалфавітні підблоки.

2. Криптоаналіз шифртексту

```
178 # --- Розділ 2: Визначення довжини ключа ---
179 print("=== ЧАСТИНА 2. ВИЗНАЧЕННЯ ДОВЖИНИ КЛЮЧА (ВАРІАНТ 3) ===")
180 ct = VARIANT_3_CIPHERTEXT
181 print(f"Довжина шифртексту: {len(ct)}")
182 print(f"I(Y) шифртексту: {index_of_coincidence(ct):.5f}\n")
183
184 print(f"{'r':<4} | {'D_r':<12} | {'Середній I_r':<12} | {'Пік'}")
185 print("-" * 40)
186
187 # Попіг відсікання шуму
188 threshold = (i_theor + i_rand) / 2
189
190 stats = []
191 peaks = []
192 for r in range(2, 31):
193     dr = coincidence_count(ct, r)
194     avg_ic = avg_block_ic(ct, r)
195
196     if avg_ic >= threshold:
197         marker = "<-- ПІК"
198         peaks.append(r)
199     else:
200         marker = ""
201
202     stats.append((r, dr, avg_ic))
203     print(f"{'r':<4} | {'dr':<12} | {'avg_ic':.5f} | {'marker'}")
204 print("-" * 40)
205
206 # Першим обирається найменший виявлений пік
207 if peaks:
208     best_r = peaks[0]
209 else:
210     best_r = max(stats, key=lambda x: x[2])[0]
211
212 print(f"Обраний період: r = {best_r}\n")
213
```

Для визначення довжини невідомого ключа було перевірено значення довжини r від 2 до 30 за двома методами:

Підрахунок кількості збігів однакових символів на відстані r (статистика D_r).

Розрахунок середнього індексу відповідності підблоків I_r .

```
=== ЧАСТИНА 2. ВИЗНАЧЕННЯ ДОВЖИНИ КЛЮЧА (ВАРІАНТ 3) ===  
Довжина шифртексту: 5224  
I(Y) шифртексту: 0.03374
```

r	D_r	Середній I_r	Пік
2	172	0.03383	
3	156	0.03370	
4	191	0.03381	
5	158	0.03361	
6	193	0.03371	
7	137	0.03732	
8	164	0.03371	
9	147	0.03387	
10	174	0.03367	
11	201	0.03378	
12	163	0.03377	
13	154	0.03392	
14	318	0.04454	<-- ПІК
15	169	0.03334	
16	153	0.03370	
17	168	0.03371	
18	177	0.03385	
19	175	0.03353	
20	131	0.03383	
21	159	0.03733	
22	161	0.03392	
23	164	0.03362	
24	169	0.03383	
25	168	0.03368	
26	166	0.03404	
27	170	0.03380	
28	308	0.04462	<-- ПІК
29	139	0.03369	
30	137	0.03332	

```
Обраний період: r = 14
```

За результатами аналізу таблиці було виділено три основні кандидати:

Кандидат $r = 14$:

Кількість збігів: $D_r = 318$ (абсолютний максимум серед усіх значень).

Середній індекс відповідності: $I_r = 0,04454$ (різкий сплеск вгору від фонового шуму 0,033).

Це мінімальне значення r , на якому спостерігається одночасний стрибок обох статистик.

Кандидат $r = 28$:

Кількість збігів: $D_r = 308$ (другий за величиною пік).

Середній індекс відповідності: $I_r = 0,04462$ (дуже близький до $r = 14$).

Число 28 є кратним до 14 ($28 = 14 * 2$). При поліалфавітному шифруванні будь-який період, що ділиться на справжню довжину ключа, також показує високий індекс, оскільки символи з таким кроком зашифровані тими самими літерами ключа.

Кандидат $r = 7$:

Кількість збігів: $D_r = 137$ (на рівні шуму).

Середній індекс відповідності: $I_r = 0,03732$ (невелике локальне підвищення).

Число 7 є дільником 14, проте значення D_r не дає сплеску. Це означає, що період 7 є помилковим і не відповідає реальному ключу.

Обґрунтування вибору кінцевого періоду:

Хоча для $r = 28$ середній індекс відповідності незначно вищий через похибку розрахунку на менших блоках, істинною довжиною ключа обирається $r = 14$.

Головне правило криптоаналізу полягає у виборі найменшого спільного періоду. Якщо обрати $r = 28$, ключ просто повторить те саме слово двічі. Тому шукана довжина ключа дорівнює 14

3. Відновлення ключа та відкритого тексту

```
214 # --- Розділ 3: Відновлення ключа та дешифрування ---
215 print(f"=== ЧАСТИНА 3. ВІДНОВЛЕННЯ КЛЮЧА ТА ТЕКСТУ (r = {best_r}) ===")
216 recovered_key = []
217 for i in range(best_r):
218     col = ct[i::best_r]
219     n_col = len(col)
220     counts = Counter(col)
221
222     best_k = 0
223     best_score = -1.0
224
225     for k in range(M):
226         score = 0.0
227         for ch, p in rus_freqs.items():
228             x_idx = CHAR_TO_IDX[ch]
229             cipher_idx = (x_idx + k) % M
230             cipher_ch = IDX_TO_CHAR[cipher_idx]
231             score += (counts.get(cipher_ch, 0) / n_col) * p
232
233         if score > best_score:
234             best_score = score
235             best_k = k
236
237     recovered_key.append(IDX_TO_CHAR[best_k])
238
239 key_str = "".join(recovered_key)
240 print(f"Відновлений ключ: {key_str}")
241
242 decrypted = vigenere_decrypt(ct, key_str)
243 print(f"Фрагмент розшифрованого тексту:\n{decrypted[:200]}\n")
244
245 with open("decrypted_variant3.txt", "w", encoding="utf-8") as f:
246     f.write(decrypted)
247
```

Після визначення довжини ключа ($r = 14$) криптоаналіз шифру Віженера зводиться до паралельного зламу 14 незалежних шифрів Цезаря. Для цього шифртекст було розбито на 14 підблоків (стовпців). Для кожного i -го підблоку було протестовано всі 32 можливі зсуви k і розраховано кореляцію отриманих частот із частотами відкритого тексту, отриманими в комп'ютерному практикумі №1. Зсув із максимальною кореляцією визначає i -ту літеру ключа.

```
=== ЧАСТИНА 3. ВІДНОВЛЕННЯ КЛЮЧА ТА ТЕКСТУ (r = 14) ===
Відновлений ключ: экомаятникфуко
Фрагмент розшифрованого тексту:
итутяувиделмаятникшарвисящийнадолгойнитипущеннойсвольтыхоравизо
пределенотношенииэквдратногокорнядлины
```

Проміжні статистичні результати відновлення літер ключа:

Позиція в ключі	Оптимальний зсув k	Відновлена літера	Кореляція з мовою
0	30	э	0,04712
1	11	к	0,04523
2	15	о	0,04981
3	13	м	0,04634
4	0	а	0,04856
5	31	я	0,04419
6	19	т	0,04780
7	14	н	0,04652
8	9	и	0,04891
9	11	к	0,04587
10	21	ф	0,04403
11	20	у	0,04677
12	11	к	0,04511
13	15	о	0,04905

Зіставлення відновлених символів дає змістовний ключ довжиною 14 символів: «экомаятникфуко»

Отриманий текст є осмисленим, граматично правильним фрагментом твору російською мовою. Це є абсолютним підтвердженням правильності визначеної довжини періоду $r = 14$ та всіх відновлених літер ключа.

4. Дослідження впливу помилки у ключі

```
247
248 # --- Розділ 4: Дослідження впливу помилки в ключі ---
249 print("=== ЧАСТИНА 4. ДОСЛІДЖЕННЯ ПОМИЛКИ В КЛЮЧІ ===")
250 err_char = "a" if key_str[0] != "a" else "b"
251 corrupted_key = err_char + key_str[1:]
252 dec_err = vigenere_decrypt(ct, corrupted_key)
253
254 diffs = [i for i in range(len(ct)) if decrypted[i] != dec_err[i]]
255 print(f"Правильний ключ: {key_str}")
256 print(f"Ключ з помилкою: {corrupted_key}")
257 print(
258     f"Помилкових символів: {len(diffs)} / {len(ct)} ({len(diffs)/len(ct)*100:.2f}%)
259 ")
260 print(f"Періодичність помилки: кожен {best_r}-й символ (i % {best_r} == 0)")
261
262
263 if __name__ == "__main__":
264     main()
```

Попередній прогноз до проведення експерименту:

Оскільки шифр Віженера є посимвольним і ключ застосовується циклічно з періодом $r = 14$, заміна одного символу ключа не призведе до повного руйнування всього тексту. Очікую такі наслідки:

Періодичність помилок: Спотвореними виявляться лише ті символи відкритого тексту, які дешифруються зміненою літерою ключа (кожен 14-й символ тексту з позиціями 0, 14, 28, 42 тощо).

Відсоток спотворень: Частка помилкових символів у розшифрованому тексті становитиме рівно $1/14$ від загального обсягу, тобто приблизно 7,14%.

Читабельність тексту: Решта 13 із 14 символів залишаться розшифрованими абсолютно правильно, тому загальний зміст тексту все одно можна буде легко зрозуміти людині.

```
=== ЧАСТИНА 4. ДОСЛІДЖЕННЯ ПОМИЛКИ В КЛЮЧІ ===
Правильний ключ: экомаятникфуко
Ключ з помилкою: акомаятникфуко
Помилкових символів: 374 / 5224 (7.16%)
Періодичність помилки: кожен 14-й символ (i % 14 == 0)
PS C:\Users\Lenovo\Desktop\ФБ43_цвілій>
```

Результати експерименту:

У ході експерименту перший символ правильного ключа «экомаятникфуко» було замінено на літеру «а»

Порівняння з прогнозом:

Прогнозована частка помилок: 1/14 (близько 7,14%)	Фактична частка помилок: 374 символи з 5224, що становить 7,16%(різниця зумовлена тим, що 5224 не ділиться націло на 14)
--	---

помилка проявилася строго на позиціях, кратних довжині ключа (кожен 14-й символ), що повністю відповідає прогнозу

Пояснення отриманого ефекту:

У шифрі Віженера кожен символ шифртексту розшифровується незалежно від сусідніх за формулою простого додавання за модулем. Через це лавинний ефект повністю відсутній: спотворення однієї літери ключа не ламає сусідні літери. Усі інші 13 позицій ключа продовжують працювати коректно, завдяки чому 92,84% тексту залишаються неушкодженими. Людина без зусиль може прочитати такий текст і навіть здогадатися, яка саме літера ключа була введена неправильно, за змістом слів.

Отже, відсутність лавинного ефекту та локальний характер поширення помилок є суттєвим криптографічним недоліком шифру Віженера, що значно полегшує його злам та ручну корекцію знайденого ключа.

1. Як змінювався індекс відповідності шифртексту при використанні ключів різної довжини? Поясніть отриманий результат.

При збільшенні довжини ключа індекс відповідності шифртексту монотонно зменшувався: від 0,05614 (для відкритого тексту) до 0,03307 (для ключа довжиною 12).

Це пояснюється тим, що шифр Віженера накладає на текст кілька різних зсувів алфавіту. Внаслідок цього однакові літери відкритого тексту перетворюються на різні символи шифртексту, що згладжує природні частотні піки мови. Зі зростанням довжини ключа розподіл літер дедалі сильніше наближається до рівномірного випадкового шуму.

2. За якими ознаками було визначено найбільш імовірну довжину невідомого ключа?

Довжину ключа було визначено за двома взаємопов'язаними статистичними ознаками:

Стрибок кількості збігів (статистика D_r): на кроках, кратних довжині ключа, значення D_r різко підскочило до 318 при фоновому рівні шуму 130–190.

Стрибок середнього індексу відповідності підблоків (I_r): індекс підблоків піднявся від рівня шуму 0,033 до 0,04454, що наближається до властивостей природної мови.

Істинною довжиною було обрано найменше значення періоду, на якому виникли ці сплески ($r = 14$).

3. Які альтернативні кандидати на довжину ключа ви досліджували, чому обрали саме їх та які результати отримали?

Було досліджено такі альтернативні значення:

$r = 28$: значення $D_r = 308$ та $I_r = 0,04462$ також показали явний пік. Цей кандидат був обраний для аналізу, оскільки число 28 кратне 14 ($28 = 14 * 2$). За таких умов символи також шифруються однаковими літерами ключа, проте це подвоєний, а не базовий період.

$r = 7$: як можливий дільник числа 14. Результат склав $D_r = 137$ та $I_r = 0,03732$, що відповідає звичайному шуму і доводить хибність цього періоду.

Остаточню обрано $r = 14$ як найменший спільний період.

4. Яка статистична інформація використовувалася для відновлення окремих символів ключа?

Використовувався емпіричний одновимірний розподіл частот літер російської мови, розрахований на великому масиві відкритого тексту в комп'ютерному практикумі 1. Для кожного з 14 стовпців шифртексту перевірялися всі 32 зсуви, і обирався той, який давав найкращий збіг частот стовпця з еталонними частотами мови.

5. Як можна перевірити правильність відновленого ключа, не знаючи початкового відкритого тексту?

Змістовність ключа: у більшості практичних завдань ключем виступає осмислене слово або фраза. У нашому випадку утворилося словосполучення *экомаятникфуко*

Змістовність розшифрованого тексту: після дешифрування утворюються граматично і синтаксично правильні слова природної мови.

Індекс відповідності: індекс розшифрованого тексту різко зростає від 0,033 до природного значення мови ($\sim 0,056$).

6. Як змінився результат розшифрування після внесення помилки в один символ ключа? Поясніть характер і періодичність змін.

Помилка проявилася суворо періодично: неправильно розшифрувався лише

кожен 14-й символ тексту, $1/14$ від усього обсягу.

Решта 92,84% тексту залишилися абсолютно неушкодженими. Це свідчить про повну відсутність лавинного ефекту розмноження помилок у шифрі Віженера: помилка в одному символі ключа впливає виключно на символи, зашифровані цією конкретною позицією.

7. Які етапи проведеного криптоаналізу найбільше залежать від обсягу доступного шифртексту? Чому?

Найбільше від обсягу шифртексту залежить відновлення символів ключа (розрахунок зсувів Цезаря для кожного стовпця) та знаходження довжини ключа g .

Причина полягає в тому, що весь текст ділиться на g підблоків. Якщо шифртекст короткий, то в кожен підблок потрапляє занадто мало літер, виникають сильні випадкові відхилення, і частотний аналіз не може однозначно виділити піки частих літер.

8. Які результати ваших експериментів найкраще демонструють слабкість шифру Віженера з періодичним ключем?

Збереження періодичності: чіткі піки статистики D_r та індексу I_r дозволяють легко і без знання ключа обчислити його довжину g .

Розпад на незалежні шифри: після знаходження g складний поліалфавітний шифр повністю розпадається на g найпростіших шифрів Цезаря, кожен з яких зламується незалежно простим перебором 32 зсувів.

Відсутність лавинного ефекту: можливість відновлювати або виправляти ключ по одній літері незалежно від інших, оскільки помилка в одній позиції не заважає правильно розшифрувати решту тексту.

Висновок:

У ході виконання комп'ютерного практикуму було засвоєно методи частотного криптоаналізу поліалфавітних шифрів на прикладі шифру Віженера. За допомогою розрахунку індексу відповідності та статистики збігів було успішно визначено довжину ключа ($g = 14$), відновлено змістовний ключ «екомяятникфуко» та повністю розшифровано відкритий текст. Проведені експерименти продемонстрували низьку стійкість шифру з періодичним ключем, оскільки він легко розпадається на незалежні шифри Цезаря через збереження періодичності мови та повну відсутність лавинного ефекту.